



PasswordStore Audit Report

Version 1.0

Cyfrin.io

May 4, 2026

Protocol Audit Report

Fred Gitonga

May 4, 2026

Prepared by: Fred Gitonga Lead Auditors: - Fred Gitonga

Table of Contents

- Table of Contents
- Protocol Summary
- Disclaimer
- Risk Classification
- Audit Details
 - Scope
 - Roles
- Executive Summary
 - Issues found
- Findings
 - High
 - Informational

Protocol Summary

PasswordStore is a smart contract application that allows a single owner to store and retrieve a private password on-chain. The protocol enforces that only the deploying address (the owner) can set and read the stored password. The contract is written in Solidity 0.8.18 and is intended for deployment on an EVM-compatible blockchain.

Disclaimer

The FRED GITONGA team makes all effort to find as many vulnerabilities in the code in the given time period, but holds no responsibilities for the findings provided in this document. A security audit by the team is not an endorsement of the underlying business or product. The audit was time-boxed and the review of the code was solely on the security aspects of the Solidity implementation of the contracts.

Risk Classification

		Impact		
		High	Medium	Low
Likelihood	High	H	H/M	M
	Medium	H/M	M	M/L
	Low	M	M/L	L

Severity is determined by two factors:

- **Impact:** How severe is the outcome if the vulnerability is exploited?
 - **High:** Funds directly at risk, or severe disruption of protocol functionality. Highly probable to happen.
 - **Medium:** Funds indirectly at risk, or some disruption under specific conditions.
 - **Low:** Funds not at risk, minimal impact, unlikely to occur.
- **Likelihood:** How probable is it that the vulnerability will be exploited?

We use the CodeHawks severity matrix to determine severity. See the documentation for more details.

Audit Details

Commit Hash:

```
1 Not available – local review
```

Scope

The following contracts were in scope of the audit:

```
1  -- src/
2  -- PasswordStore.sol
```

Roles

- **Owner:** The deploying address. Only the owner is intended to call `setPassword` and `getPassword`.
- **Outsiders:** Any address that is not the owner. These users should have no ability to set or read the password.

Executive Summary

The audit of the PasswordStore protocol was conducted as a focused, time-boxed security review. The primary objective was to assess the security of the on-chain password storage mechanism, the access control implementation, and the correctness of NatSpec documentation.

During the review, two critical high-severity vulnerabilities were identified — both of which fundamentally undermine the protocol's core security guarantee — along with one informational issue relating to incorrect NatSpec documentation.

Issues found

Severity	Number of Issues
High	2
Medium	0
Low	0
Informational	1
Total	3

[H-2] PasswordStore::setPassword has no access controls — any user can change the password**Description:**

The `PasswordStore::setPassword` function is declared `external` with no access control checks. The protocol's purpose and NatSpec indicate that only the owner should be able to set a new password, but there is nothing preventing any address from calling this function.

```
1 function setPassword(string memory newPassword) external {
2     // @Audit - There are no Access Controls.
3     s_password = newPassword;
4     emit SetNewPassword();
5 }
```

Impact:

Anyone can set or change the stored password, severely breaking the contract's intended functionality and access control invariant.

Proof of Concept:

```
1 function test_anyone_can_set_password(address randomAddress) public {
2     vm.assume(randomAddress != owner);
3     vm.startPrank(randomAddress);
4     string memory expectedPassword = "myNewPassword";
5     passwordStore.setPassword(expectedPassword);
6
7     vm.startPrank(owner);
8     string memory actualPassword = passwordStore.getPassword();
9     assertEq(actualPassword, expectedPassword);
10 }
```

Recommended Mitigation:

Add an access control check to restrict `setPassword` to the contract owner:

```
1 if (msg.sender != s_owner) {
2     revert PasswordStore__NotOwner();
3 }
```

Informational

[I-1] PasswordStore::getPassword NatSpec references a non-existent parameter

Description:

The NatSpec for `getPassword` incorrectly documents a `@param newPassword` that does not exist in the function signature:

```
1  /*
2   * @notice This allows only the owner to retrieve the password.
3   * @param newPassword The new password to set.
4   */
5  function getPassword() external view returns (string memory) {}
```

The function signature is `getPassword()` with no parameters, making this NatSpec entry erroneous.

Impact:

Incorrect NatSpec reduces documentation quality and can mislead developers integrating with the contract.

Proof of Concept:

Not applicable.

Recommended Mitigation:

Remove the incorrect `@param` line from the NatSpec:

```
1  /*
2   * @notice This allows only the owner to retrieve the password.
3   - * @param newPassword The new password to set.
4   */
```